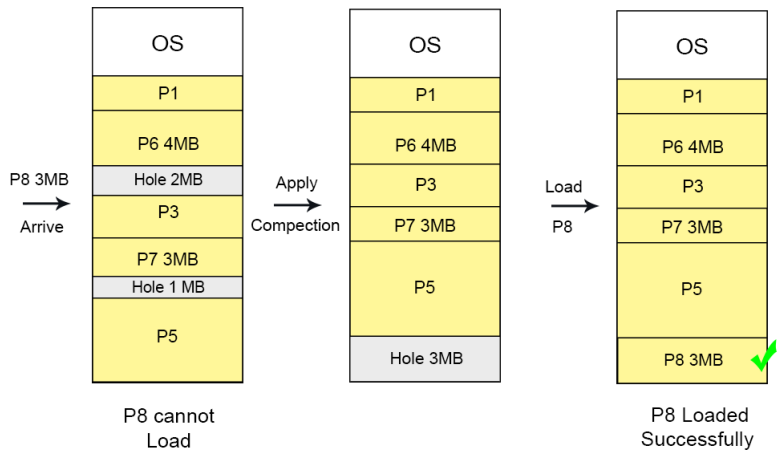
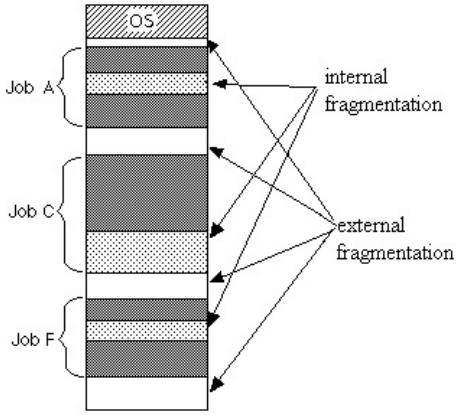



รูปที่ 7.15: การอัดแน่นเพื่อลดการสูญเสียพื้นที่ย่อยภายนอก




รูปที่ 7.16: การสูญเสียพื้นที่ย่อยภายใน

## 7.4 การแบ่งพื้นที่เป็นหน้า

การแบ่งพื้นที่หน่วยความจำเป็นหน้า หรือ*เพจจิ้ง (paging)* เป็นเทคนิคการจัดการหน่วยความจำที่ได้รับความนิยมและใช้อย่างแพร่หลาย แนวคิดหลักคือ การแบ่งพื้นที่หน่วยความจำออกเป็นส่วนย่อย ๆ ที่มีขนาดเท่ากันทุกส่วน โดยแต่ละส่วนนี้เรียกว่า*หน้า (page)*

หลักการของเพจจิ้งคือ การกำหนดให้พื้นที่หน่วยความจำหลักทั้งหมด ถูกแบ่งออกเป็นหน่วยที่มีขนาดเท่ากัน เมื่อต้องการจัดสรรพื้นที่ให้แก่โพรเซส ระบบจะจัดสรรเป็นจำนวนเต็มของหน้าเสมอ เช่น หากโพรเซสต้องการพื้นที่ 3.5 หน้า ระบบจะจัดสรรจริง 4 หน้าให้ ถึงแม้จะมีพื้นที่บางส่วนที่ไม่ได้ถูกใช้งานจริงก็ตาม ทำให้เกิดปัญหาที่เรียกว่า *internal fragmentation* ขึ้นได้

เพื่อให้โพรเซสสามารถเข้าถึงหน่วยความจำ ที่อยู่ที่ของโพรเซสใช้อ้างอิงที่เรียกว่า*ที่อยู่เสมือน (virtual address)* ซึ่งจะถูกแปลงไปเป็น*ที่อยู่กายภาพ (physical address)* ในหน่วยความจำหลัก การแปลงนี้เป็นหน้าที่ของ*หน่วยจัดการหน่วยความจำ (Memory Management Unit: MMU)* ซึ่งเป็นฮาร์ดแวร์ที่ออกแบบมาเพื่อลดภาระการทำงานของหน่วยประมวลผลกลาง (ในปัจจุบัน MMU คืออุปกรณ์ฮาร์ดแวร์ที่สำคัญในคอมพิวเตอร์ ทำหน้าที่หลักในการแปลงที่อยู่เสมือน (virtual address) ซึ่งเป็นที่อยู่ที่หน่วยประมวลผลก

ลางสร้างขึ้นมาเป็นที่อยู่กายภาพ (physical address) ที่ใช้ในการเข้าถึงหน่วยความจำหลัก (RAM))

ข้อดีของการแบ่งพื้นที่แบบเพจจิ้งคือ โพรเซสไม่จำเป็นต้องถูกเก็บไว้ในพื้นที่ต่อเนื่องกันทั้งหมด แต่สามารถถูกจัดเก็บในตำแหน่งที่กระจัดกระจายทั่วหน่วยความจำได้ ทำให้สามารถใช้พื้นที่ว่างที่มีอยู่ได้อย่างคุ้มค่า และไม่จำเป็นต้องทำการบีบอัดหรือย้ายข้อมูลเพื่อจัดเรียงหน่วยความจำใหม่ก่อนการใช้งาน

การแบ่งพื้นที่เป็นหน้าสามารถอธิบายได้ 2 ลักษณะ ดังนี้:

1. การจัดสรรหน่วยความจำทางกายภาพ (Paging model of physical memory) หน่วยความจำกายภาพจะถูกแบ่งออกเป็นบล็อกที่มีขนาดคงที่ โดยบล็อกเหล่านี้เรียกว่า**เฟรม (frames)** โดยขนาดของเฟรมถูกกำหนดโดยฮาร์ดแวร์ ซึ่งมักเป็นกำลังของ 2 เช่น 512 ไบต์, 1 KB, 4 KB, ไปจนถึง 16 MB ต่อเฟรม ทั้งนี้ขึ้นอยู่กับสถาปัตยกรรมคอมพิวเตอร์แต่ละรุ่น

2. การจัดสรรหน่วยความจำทางตรรกะ (Paging model of logical memory) หน่วยความจำเสมือน (logical memory หรือ virtual memory) จะถูกแบ่งออกเป็น**เพจ (pages)** ซึ่งมีขนาดเท่ากับเฟรมในหน่วยความจำกายภาพ การอ้างอิงที่อยู่เสมือนหนึ่งตำแหน่ง จะถูกแบ่งออกเป็นสองส่วน (ดังแสดงในรูปที่ 7.17):

    - หมายเลขเพจ (Page number: $p$) ทำหน้าที่เป็นดัชนี (index) เพื่อชี้ไปยัง*ตารางเพจ (page table)* ซึ่งตารางนี้จะบอกตำแหน่งเริ่มต้น (base address) ของเฟรมที่สอดคล้องกันในหน่วยความจำกายภาพ
    - ขอบเขตเพจ (Page offset: $d$) เป็นตำแหน่งย่อยภายในเฟรม เมื่อรวมกับตำแหน่งเริ่มต้นที่ได้จากตารางเพจ จะสามารถระบุที่อยู่จริงในหน่วยความจำทางกายภาพได้อย่างถูกต้อง



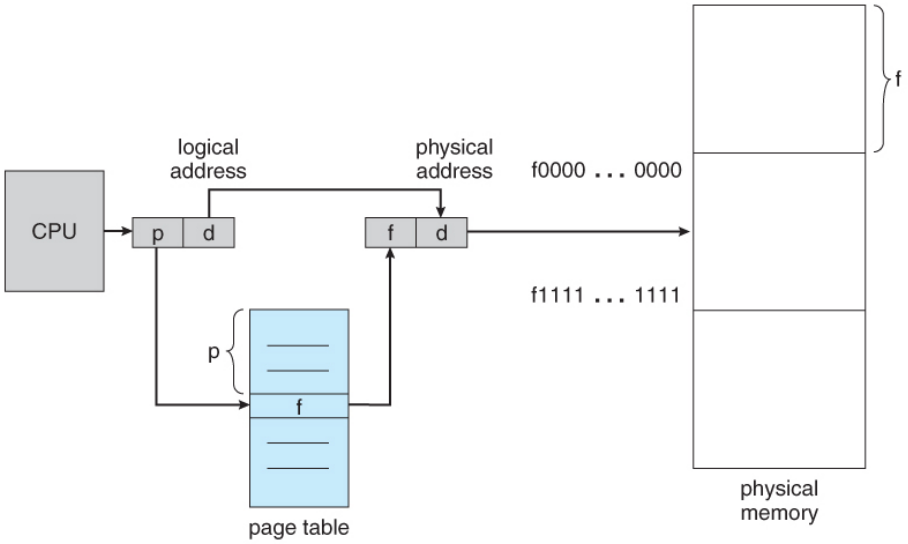

รูปที่ 7.17: การแปลงที่อยู่เสมือนไปเป็นที่อยู่กายภาพโดยใช้ตารางเพจ

### 7.4.1 การทำงานของเทคนิคเพจจิ้ง

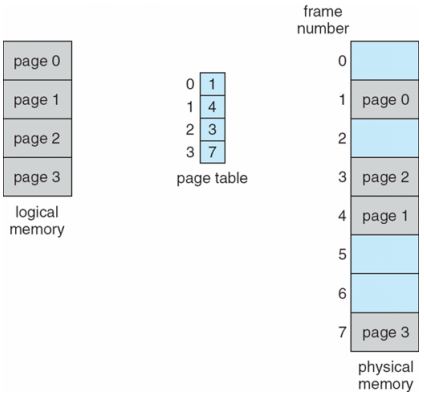
หลักการทำงานของเพจจิ้ง คือการที่โพรเซสหนึ่ง ๆ สามารถใช้จำนวนหน้าที่ไม่จำเป็นต้องเรียงต่อเนื่องกันในหน่วยความจำหลักได้ ซึ่งการจัดสรรพื้นที่ลักษณะนี้ จะไม่ทำให้เกิดช่องว่างระหว่างหน้า ดังนั้น จึงไม่มีพื้นที่ที่ถูกทิ้งไว้โดยเปล่าประโยชน์ และช่วยป้องกันไม่ให้เกิดปัญหา external fragmentation ซึ่งเป็นปัญหาสำคัญของการจัดสรรหน่วยความจำแบบต่อเนื่อง

จากรูปที่ 7.18 และ 7.19 สามารถอธิบายได้ว่าการแบ่งเพจทำงานโดยใช้ page table ซึ่งเป็นตารางที่เก็บข้อมูลการจับคู่ระหว่าง *page number* และ *frame number* โดยตารางนี้จะถูกเก็บไว้ในหน่วยความจำ ซึ่ง

- page number หมายถึง หมายเลขเพจ ซึ่งเป็นตัวเลขเสมือน (*virtual address*) ที่โปรแกรมใช้เพื่ออ้างอิงตำแหน่ง
- frame number หมายถึง หมายเลขเฟรม ซึ่งเป็นตำแหน่งกายภาพ (*physical address*) ในหน่วยความจำหลัก




รูปที่ 7.18: การทำงานของเพจจิ้ง




รูปที่ 7.19: รูปแบบการแบ่งเพจในหน่วยความจำแบบตรรกะและกายภาพ

เมื่อโพรเซสต้องการเข้าถึงข้อมูลใด ๆ ระบบจะใช้หมายเลขเพจเป็นดัชนีค้นหาตารางเพจ เพื่อหาว่าหน้านั้นถูกเก็บอยู่ในเฟรมใดของหน่วยความจำจริง จากนั้นจึงใช้ข้อมูลของ *page offset* ร่วมด้วย เพื่อหาตำแหน่งจริงที่ถูกต้อง

## 7.4.2 ฮาร์ดแวร์ในการสนับสนุนการแบ่งหน้า

ในการทำงานจริง การเข้าถึงข้อมูลในหน่วยความจำที่ใช้เพจจิ้ง จำเป็นต้องใช้เวลาในการอ้างอิงสองครั้ง คือ ครั้งแรกเพื่ออ่านค่าจาก page table และครั้งที่สองเพื่ออ่านข้อมูลจากหน่วยความจำจริง ซึ่งจะทำให้การเข้าถึงข้อมูลช้าลงได้

เพื่อแก้ปัญหานี้ จึงมีการใช้ฮาร์ดแวร์ชนิดพิเศษ ที่มีความเร็วสูงและสามารถเก็บข้อมูลการแม็ปเพจบางส่วนไว้ล่วงหน้าได้ ฮาร์ดแวร์นี้เรียกว่า *Translation Look-aside Buffer (TLB)* ซึ่งเป็นหน่วยความจำขนาดเล็กที่มีความเร็วสูง ทำหน้าที่เป็นแคชสำหรับตารางเพจ

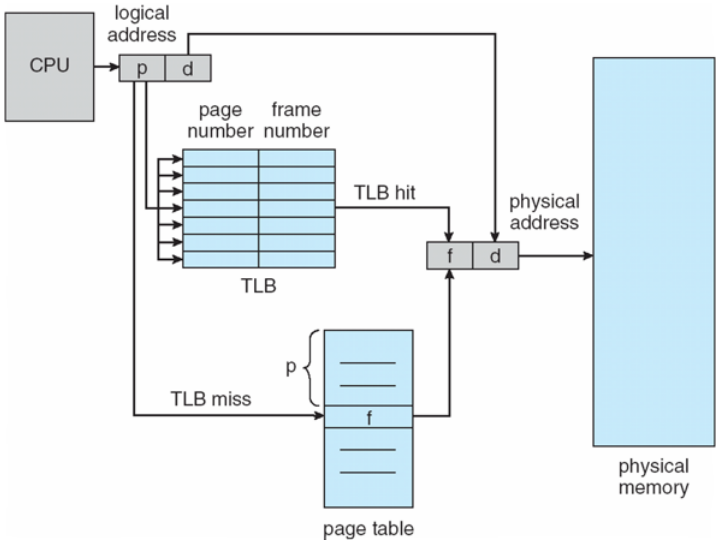
ข้อมูลใน TLB แบ่งออกเป็นสองส่วนหลัก ได้แก่

1. ส่วนเก็บกุญแจ (*Key* หรือ *Tag*)

2. ส่วนเก็บค่า (*Value*)

เมื่อมีการเรียกใช้งานข้อมูล ระบบจะตรวจสอบว่า หมายเลขเพจที่ต้องการมีอยู่ใน TLB หรือไม่ ถ้ามี (*TLB hit*) ข้อมูลจะถูกส่งกลับได้อย่างรวดเร็ว แต่ถ้าไม่มี (*TLB miss*) ระบบจะต้องค้นหาข้อมูลจากตารางเพจในหน่วยความจำจริง ซึ่งใช้เวลานานกว่า

ข้อจำกัดคือ TLB มีราคาสูงและไม่สามารถเก็บข้อมูลได้มากนัก โดยทั่วไปจะเก็บได้เพียง 64 ถึง 1024 รายการ ในกรณีที่ TLB เต็ม ระบบปฏิบัติการจำเป็นต้องเลือกเพจบางรายการออก (flushed หรือ erased) แล้วแทนที่ด้วยเพจที่ถูกใช้งานล่าสุด แต่ในบางกรณี เช่น เพจที่เก็บคำสั่งของแกนกลางระบบปฏิบัติการ (kernel code) อาจถูกกำหนดให้ไม่สามารถแทนที่ได้ ปรากฏการณ์นี้เรียกว่า *wired down* (การตรึงข้อมูลในหน่วยความจำ)




รูปที่ 7.20: ฮาร์ดแวร์กับการสนับสนุนการแบ่งหน้าด้วย TLB (Translation Look-aside Buffer)

ประสิทธิภาพของ TLB สามารถวัดได้จาก *อัตราการพบ (Hit ratio)* ซึ่งหมายถึงเปอร์เซ็นต์ของเวลาที่ค้นหาหมายเลขเพจแล้วพบใน TLB เช่น หากอัตราการพบอยู่ที่ 80% หมายความว่า 80% ของการค้นหาสามารถตอบสนองได้ทันทีจาก TLB และอีก 20% ต้องไปค้นหาต่อในหน่วยความจำจริง

ตัวอย่างเช่น หากการค้นหาใน TLB ใช้เวลา 20 นาโนวินาที และการเข้าถึงหน่วยความจำจริงใช้เวลา 100 นาโนวินาที:

- ถ้าพบใน TLB (Hit) เวลาที่ใช้รวม = 20 + 100 = 120 ns
- ถ้าไม่พบใน TLB (Miss) เวลาที่ใช้รวม = 20 + 100 + 100 = 220 ns

ดังนั้น เวลาเฉลี่ยที่ใช้ในการเข้าถึงหน่วยความจำ (*Effective Memory Access Time (EMAT)*) สามารถคำนวณได้ดังนี้:

$$\text{EMAT} = (0.80 \times 120) + (0.20 \times 220) = 140\text{ns} \tag{7.1}$$

จากการคำนวณ พบว่าเวลาเข้าถึงหน่วยความจำเพิ่มขึ้น 40% เมื่อเทียบกับเวลามาตรฐาน 100 ns แต่หากอัตราการพบเพิ่มขึ้นเป็น 98% การคำนวณจะเป็นดังนี้:

$$\text{EMAT} = (0.98 \times 120) + (0.02 \times 220) = 122\ \text{ns} \tag{7.2}$$

เมื่ออัตราการพบของ TLB สูงขึ้น เวลาการเข้าถึงหน่วยความจำโดยรวมก็จะใกล้เคียงกับเวลามาตรฐานมากขึ้น ในกรณีนี้ เวลาล่าช้าลดลงเหลือเพียง 22% จากเวลาปกติ

### 7.4.3 การป้องกันหน่วยความจำในเพจจิ้ง

ในการทำงานของระบบที่ใช้เพจจิ้ง ตารางเพจ (page table) ไม่เพียงทำหน้าที่เก็บข้อมูลการแม็ประหว่างหมายเลขเพจกับหมายเลขเฟรมเท่านั้น แต่ยังมีการเก็บ*บิตพิเศษเพื่อการป้องกัน (protection bits)* ที่ใช้ในการตรวจสอบและกำหนดสิทธิ์การใช้งานของแต่ละเพจด้วย บิตเหล่านี้จะถูกกำหนดให้กับทุกเฟรมในหน่วยความจำหลัก เพื่อช่วยควบคุมการเข้าถึงข้อมูลว่าเพจใดสามารถทำการอ่าน เขียน หรือประมวลผลได้

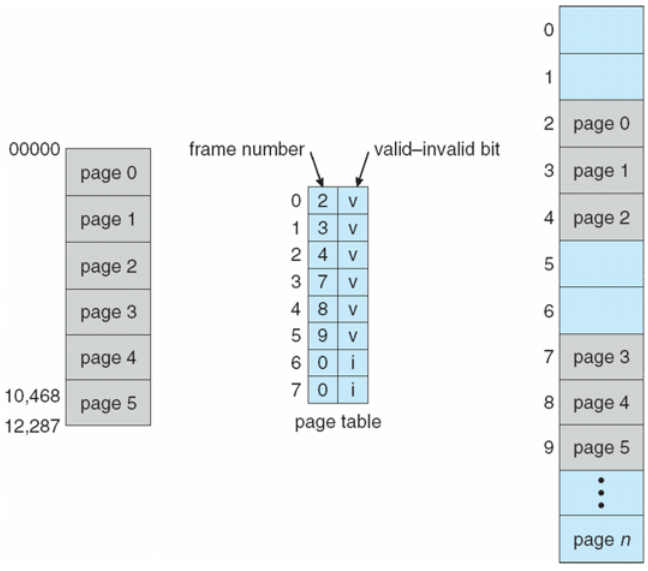
บิตสถานะที่สำคัญสามารถแบ่งออกได้เป็น 2 ประเภท คือ:

1. *บิตใช้งานได้ (Valid bit)* เป็นตัวบ่งบอกว่าเพจนั้นมีข้อมูลที่ถูกต้องและถูกโหลดเข้าสู่หน่วยความจำทางกายภาพแล้ว สามารถเข้าถึงและใช้งานได้ทันที หากโปรแกรมอ้างอิงเพจนี้ หน่วยประมวลผลกลางจะสามารถเข้าถึงข้อมูลในเฟรมที่สอดคล้องได้โดยตรง

2. *บิตใช้งานไม่ได้ (Invalid bit)* เป็นตัวบ่งบอกว่าเพจนั้นไม่สามารถใช้งานได้ในขณะนั้น ซึ่งอาจเกิดขึ้นได้จากหลายสาเหตุ เช่น

   - ระบบปฏิบัติการยังไม่ได้โหลดข้อมูลของเพจเข้าสู่หน่วยความจำหลัก
   - ข้อมูลเพจถูกสลับออก (swapped out) ไปยังหน่วยความจำสำรอง

   หากโปรแกรมพยายามเข้าถึงเพจที่มีบิตเป็น Invalid จะทำให้เกิดข้อผิดพลาดที่เรียกว่า *page fault* เมื่อเกิด page fault ขึ้น ระบบจะต้องทำการโหลดเพจที่ต้องการจากหน่วยความจำสำรองเข้าสู่หน่วยความจำหลัก และปรับสถานะจาก Invalid ให้เป็น Valid ก่อนที่จะดำเนินการประมวลผลต่อ

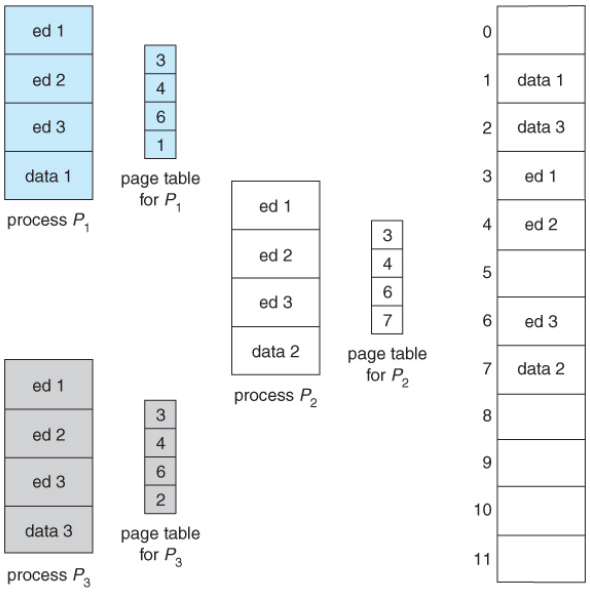



รูปที่ 7.21: ตารางเพจของบิตที่ใช้งานได้และไม่ได้ (Valid ($v$) หรือ Invalid ($i$) bit ในตารางเพจ)

### 7.4.4 การใช้เพจร่วมกัน

นอกจากการป้องกันหน่วยความจำแล้ว ระบบปฏิบัติการยังรองรับแนวคิด*การใช้เพจร่วมกัน (page sharing)* ซึ่งเป็นกลไกสำคัญที่ช่วยเพิ่มประสิทธิภาพในการใช้หน่วยความจำ การแบ่งปันเพจมักใช้กับข้อมูลหรือโปรแกรมที่ไม่ถูกแก้ไขระหว่างการประมวลผล (*non-self-modifying code*) ข้อมูลลักษณะนี้เรียกว่า *Reentrant Code* หรือ *Pure Code*

รหัสคำสั่งที่สามารถแชร์ได้จะถูกเก็บไว้ในตำแหน่งทางตรรกะเดียวกัน และโพรเซสทุกตัวที่ต้องการใช้งานรหัสคำสั่งนั้นจะชี้มายังเพจเดียวกัน ทำให้ไม่จำเป็นต้องมีสำเนาของรหัสคำสั่งแยกเก็บในแต่ละโพรเซส ในขณะที่เพจข้อมูล (*data page*) แต่ละโพรเซสยังคงมีเป็นของตัวเอง




รูปที่ 7.22: การใช้งานเพจร่วมกัน (Sharing of Code in a Paging Environment)

จากรูปที่ 7.22 แสดงตัวอย่างการใช้เพจร่วมกันระหว่างโพรเซส $P_1$, $P_2$, และ $P_3$ ซึ่งทั้งหมดต้องการใช้งานโปรแกรม *Text Editor* ร่วมกัน โปรแกรมนี้ประกอบด้วยเพจที่ใช้เก็บรหัสคำสั่ง (Page 0, Page 1, Page 3) ซึ่งถูกเก็บในเฟรมที่ 3, 4 และ 6 ของหน่วยความจำจริง ดังนั้น โพรเซสทั้งสามจะชี้มายังเฟรมเหล่านี้ร่วมกัน แต่ยังคงมีเพจข้อมูลเป็นของตัวเอง

เพื่อให้เห็นความแตกต่าง สมมุติว่ามีผู้ใช้ 40 คน ที่ต้องการใช้โปรแกรม *Text Editor* เดียวกัน โปรแกรมมีขนาด 150 KB และแต่ละโพรเซสต้องใช้พื้นที่ข้อมูลส่วนตัว 50 KB

- หากไม่มีการใช้เพจร่วมกัน จะต้องใช้หน่วยความจำรวม

$$40 \times (150 + 50) = 8{,}000 \text{ KB}$$

- แต่หากใช้กลไกการแชร์เพจ รหัสคำสั่ง 150 KB จะถูกเก็บเพียงครั้งเดียวและแชร์ให้ผู้ใช้ทั้งหมด ดังนั้น พื้นที่หน่วยความจำที่ต้องใช้จริง จะลดลงอย่างมาก เหลือเพียง

$$150 + (40 \times 50) = 2{,}150 \text{ KB}$$

ตัวอย่างนี้ชี้ให้เห็นว่า การใช้เพจร่วมกันช่วยประหยัดหน่วยความจำอย่างมีนัยสำคัญ โดยเฉพาะอย่างยิ่งในระบบที่มีผู้ใช้งานหลายคน ซึ่งเป็นแนวคิดพื้นฐานของระบบ *time sharing systems*

### 7.4.5 โครงสร้างของตารางเพจ

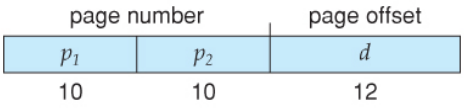
ตารางเพจ (page table) เป็นโครงสร้างข้อมูลที่ใช้ในการแม็ประหว่าง*ที่อยู่เสมือน (Virtual Address)* และ*ที่อยู่กายภาพ (Physical Address)* การออกแบบโครงสร้างตารางเพจมีหลายวิธีเพื่อแก้ปัญหาความซับซ้อนและขนาดของตาราง โดยทั่วไปสามารถแบ่งออกเป็น 3 รูปแบบ ได้แก่:

#### 1. โครงสร้างแบบลำดับชั้น (Hierarchical Paging)

ระบบคอมพิวเตอร์สมัยใหม่รองรับพื้นที่ทางตรรกะ (logical-address space) ที่มีขนาดใหญ่มาก ตั้งแต่ $2^{32}$ ถึง $2^{64}$ ไบต์ หากใช้ตารางเพจแบบปกติ ตารางเพจจะมีขนาดใหญ่มากจนไม่สามารถจัดเก็บได้อย่างมีประสิทธิภาพ ดังนั้น จึงมีการแบ่งโครงสร้างตารางเพจออกเป็นหลายระดับ เพื่อให้สามารถจัดการได้ง่ายขึ้น

วิธีที่นิยมคือ *การแบ่งเพจแบบสองระดับ (two-level page table)* ซึ่งโครงสร้างของที่อยู่เสมือนจะประกอบด้วย 2 ส่วน ได้แก่:

1. *หมายเลขเพจ (page number)* ใช้เพื่อค้นหาตำแหน่งในตารางเพจระดับแรกและระดับที่สอง
2. *ขอบเขตเพจ (page offset)* ใช้ระบุตำแหน่งย่อยภายในเฟรมจริงที่แม็ปไว้

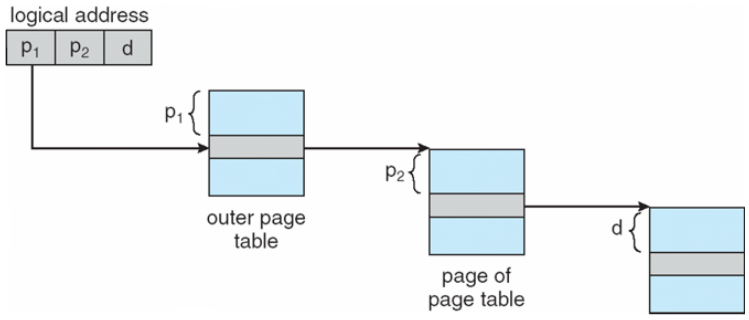



รูปที่ 7.23: การแบ่งเพจแบบ 2 ระดับ – หมายเลขเพจและขอบเขตเพจ

**ตัวอย่าง:** ในระบบคอมพิวเตอร์แบบ 32 บิต ที่มีขนาดเพจ (page size) เท่ากับ 4 KB ที่อยู่เสมือนจะถูกแบ่งออกเป็น 2 ส่วนคือ:

- ส่วนแรกขนาด 20 บิต ใช้แทนหมายเลขเพจ (page number)
- ส่วนที่สองขนาด 12 บิต ใช้แทนขอบเขตเพจ (page offset)

ดังที่แสดงในรูปที่ 7.23 และ 7.24




รูปที่ 7.24: โครงสร้างการแปลงเลขที่อยู่สำหรับสถาปัตยกรรมแบบ two-level 32-bit

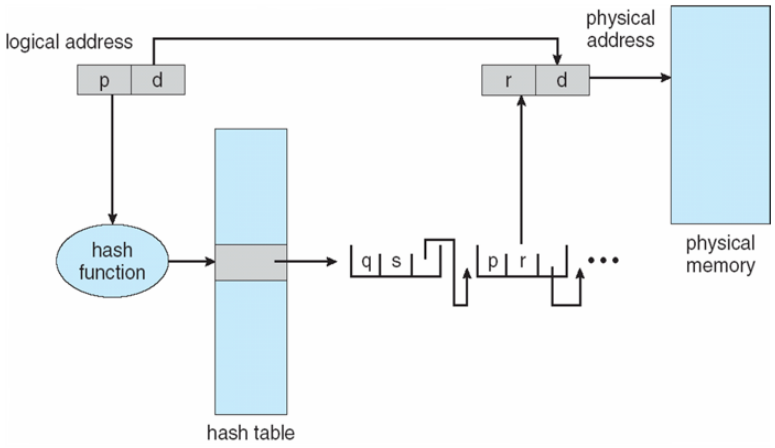
### 2. โครงสร้างแบบตารางแฮช (Hash Page Table)

สำหรับระบบคอมพิวเตอร์ที่รองรับพื้นที่หน่วยความจำเสมือนขนาดใหญ่มากกว่า 32 บิต การใช้ตารางเพจแบบลำดับชั้นอาจไม่เพียงพอ จึงมีการใช้*ตารางแฮช (Hash page table)*

หลักการคือ หมายเลขเพจเสมือน (virtual-page number) จะถูกป้อนเข้าสู่ฟังก์ชันแฮช เพื่อให้ได้ตำแหน่งที่อยู่ในตารางแฮช แต่ละตำแหน่งในตารางแฮชจะประกอบด้วยลิงก์ลิสต์ (linked list) เพื่อรองรับกรณีที่ผลลัพธ์การแฮชซ้ำกัน (collision)

ข้อมูลในแต่ละโหนดของลิงก์ลิสต์ประกอบด้วย:

- ค่าหมายเลขเพจเสมือน (virtual-page number)
- ค่าดัชนีที่ชี้ไปยังเฟรมเพจ (page frame)
- ค่าพอยเตอร์ที่ชี้ไปยังโหนดถัดไปในลิงก์ลิสต์




รูปที่ 7.25: โครงสร้างแบบตารางแฮช (Hashed page table)

### 3. โครงสร้างเพจแบบผกผัน (Inverted Page Table)

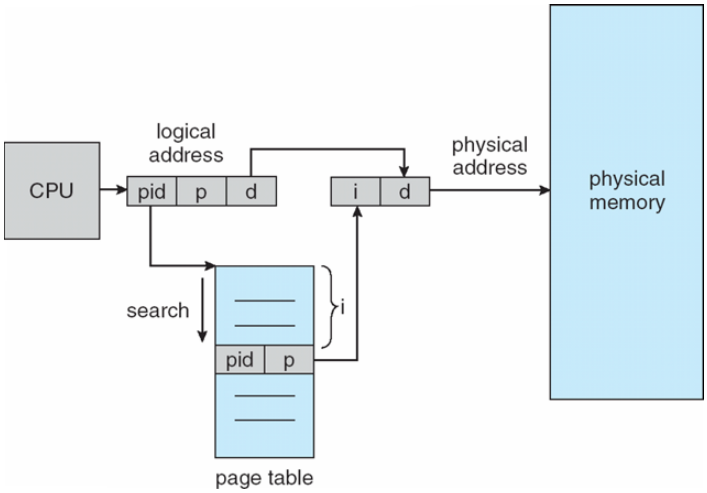
*เพจแบบผกผัน (Inverted Page Table)* เป็นเทคนิคที่ใช้ลดขนาดของตารางเพจ โดยไม่สร้างตารางเพจสำหรับทุกโพรเซส แต่จะมีเพียง*ตารางเพจเดียวในระบบ*

ในโครงสร้างนี้ ในแต่ละเฟรมของหน่วยความจำกายภาพ จะมีรายการในตารางเพจเพียงหนึ่งรายการที่บอกว่าเฟรมนั้นถูกแม็ปกับเพจเสมือนใด แต่ละเอนทรีของตารางเพจจะประกอบด้วยข้อมูล 3 ส่วน ได้แก่:

$$\langle process\text{-}id, page\text{-}number, offset \rangle$$

วิธีการนี้ สามารถมองในลักษณะที่ว่า page table ปกติ: คล้ายกับ “สมุดโทรศัพท์” ที่มีรายชื่อทุกบ้านในเมือง (แม้บ้านจะว่างเปล่า) แต่ Inverted page table: คล้ายกับ “สมุดโทรศัพท์” ที่มีรายชื่อเฉพาะบ้านที่มีคนอยู่จริง ทำให้สมุดบางกว่า ไม่เปลืองพื้นที่ นั่นเอง ซึ่งคือการกลับมุมมอง โดยที่จากเดิมที่ถามว่า “virtual page นี้อยู่ที่เฟรมไหน?”เปลี่ยนเป็น “เฟรมนี้กำลังถือ virtual page ไหนอยู่?

เพจแบบผกผันสามารถแสดงได้ดังแสดงในรูปที่ 7.26 วิธีนี้ช่วยลดการใช้หน่วยความจำสำหรับเก็บตารางเพจ แต่การค้นหาจะซับซ้อนขึ้น เนื่องจากต้องตรวจสอบว่าเพจเสมือนของโพรเซสใด ถูกแม็ปกับเฟรมใดในตารางเพจผกผัน




รูปที่ 7.26: โครงสร้างแบบผกผัน (Inverted page table)

### 7.4.6 เพจจิ้งและการแก้ไขปัญหา external fragmentation

ข้อดีของเพจจิ้งคือการจัดการหน่วยความจำที่ช่วยแก้ปัญหา external fragmentation ได้อย่างมีประสิทธิภาพ การจองพื้นที่หน่วยความจำจะถูกจองเป็นจำนวนเต็มของเพจเสมอ โดยไม่ทิ้งช่องว่างระหว่างหน้า

**ตัวอย่าง:** หาก 1 page มีขนาด 1,024 ไบต์ และต้องการเก็บข้อมูลขนาด 9,555 ไบต์ ระบบจะต้องจองพื้นที่เป็น 10 Page เมื่อใช้งานเสร็จและคืนพื้นที่ 3 Page พื้นที่ทั้ง 3 Page จะสามารถนำกลับมาใช้ต่อได้ทันที ดังนั้น จึงไม่เกิดปัญหา external fragmentation แม้จะเหลือช่องว่างใน page บางส่วนก็ตาม (มันจะเป็นเพจสุดท้าย)

อย่างไรก็ตาม แม้เพจจิ้งจะช่วยแก้ปัญหาการแตกภายนอก (external fragmentation) ได้ แต่กลับทำให้เกิดปัญหาใหม่ที่เรียกว่า internal fragmentation แทน

### 7.4.7 เพจจิ้งและการแก้ไขปัญหา internal fragmentation

Internal fragmentation เกิดขึ้นเมื่อขนาดข้อมูลไม่พอดีกับขนาดเพจ ทำให้มีพื้นที่ในเพจที่ไม่ได้ใช้งานจริงและถูกทิ้งไว้เป็นช่องว่างภายใน

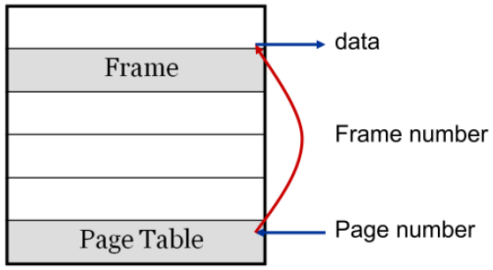
**ตัวอย่าง:** หากขนาดเพจ = 1,024 ไบต์ และข้อมูลขนาด 9,555 ไบต์ จะต้องใช้ 10 Page ซึ่งมีพื้นที่รวม 10,240 ไบต์ ทำให้เกิดพื้นที่สูญเปล่า (wasted space) จำนวน 685 ไบต์

โดยทั่วไป ปัญหานี้อาจทำให้สูญเสียพื้นที่ประมาณครึ่งหนึ่งของเพจสุดท้ายเสมอ ดังนั้น การเลือกขนาดเพจที่เหมาะสมจึงสำคัญมาก ถ้าเลือกเพจเล็กเกินไป ตารางเพจจะมีขนาดใหญ่และการค้นหาจะช้าลง แต่ถ้าเลือกเพจใหญ่เกินไป จะทำให้เกิดการสูญเสียพื้นที่มากขึ้น

ในระบบเพจจิ้งโดยปกติแล้ว การเข้าถึงหน่วยความจำแต่ละครั้ง ต้องใช้เวลาอ่านสองครั้ง ได้แก่:

1. อ่านตารางเพจเพื่อหาหมายเลขเฟรม
2. อ่านข้อมูลจริงจากเฟรมนั้น

โดยปกติ การค้นหาตำแหน่งเฟรมในตารางเพจทำแบบ *linear search หรือ sequential search* ซึ่งแม้จะง่ายต่อการออกแบบ แต่ถ้าตารางมีขนาดใหญ่มาก จะทำให้การค้นหาล่าช้าและกระทบต่อประสิทธิภาพของระบบได้




รูปที่ 7.27: การค้นหาเฟรมเพื่อทำงานกับข้อมูลที่อยู่ในเฟรมนั้น




รูปที่ 7.28: การเปรียบเทียบ page number กับตาราง page table แบบ sequential search

## 7.5 การแบ่งส่วน

การแบ่งส่วน (Segmentation) เป็นเทคนิคการจัดการหน่วยความจำที่ทำให้หน่วยความจำทางตรรกะ –ตามมุมมองของผู้เขียนโปรแกรม –เหมือนกับหน่วยความจำทางกายภาพที่มีอยู่จริงในเครื่อง ระบบปฏิบัติการจึงต้องทำหน้าที่*จับคู่ (mapping)* และแปลงตำแหน่งทางตรรกะให้เป็นตำแหน่งจริงเสมอ เพื่อให้โปรแกรมทำงานได้ถูกต้อง แม้ทั้งสองมุมมองจะต่างกันก็ตาม

### 7.5.1 แนวคิดพื้นฐาน

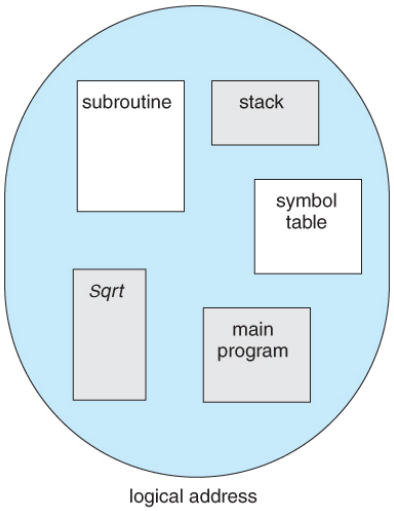
แนวคิดหลักคือแบ่งโปรแกรมออกเป็น*ส่วน (segment)*) หลาย ๆ ส่วน โดยแต่ละส่วนมีขนาด*ไม่เท่ากัน* โดยสอดคล้องกับองค์ประกอบเชิงตรรกะของโปรแกรม เช่น โปรแกรมหลัก ฟังก์ชัน ตารางสัญลักษณ์ สแต็กอาร์เรย์ และชุดตัวแปร เป็นต้น ส่วนต่าง ๆ นี้สัมพันธ์และเรียกใช้กันได้ดังรูปที่ 7.29

การอ้างอิงตำแหน่งในระบบแบ่งส่วนใช้รูปแบบ ⟨เลขส่วน (segment-number), ระยะจากขอบ (offset)⟩ ซึ่งสะท้อนโครงสร้างโปรแกรมตามที่ผู้ใช้เห็น โดยแตกต่างจากการแบ่งเป็นหน้า (paging) ที่ผู้ใช้มองเห็นเป็น*เลขเดียว* แต่ฮาร์ดแวร์ไปแยกหน้า/ออฟเซ็ตให้เองเบื้องหลัง

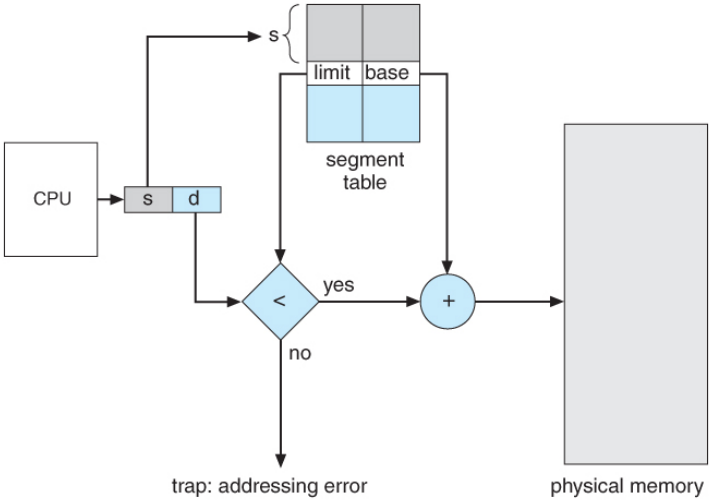
โดยทั่วไป ตัวแปลภาษาและเครื่องมือพัฒนา (เช่น assembler หรือ compiler) จะแยกโปรแกรมเป็นส่วน ๆ อัตโนมัติ เช่น ส่วนตัวแปรร่วม (global), ส่วนสแต็กสำหรับการเรียกโปรแกรมย่อย, ส่วนคำสั่งของโปรแกรมย่อย และส่วนตัวแปรภายใน (local) เป็นต้น

### 7.5.2 การแบ่งส่วนของหน่วยความจำ

แม้ผู้ใช้จะอ้างอิงตำแหน่งแบบสองมิติ (เลขส่วน + ออฟเซ็ต) แต่หน่วยความจำจริงเป็นอาเรย์หนึ่งมิติของไบต์/คำ (word) ที่เรียงต่อกัน จึงต้องมีโครงสร้างข้อมูลเพื่อทำการจับคู่สองมิติ → หนึ่งมิติ ซึ่งก็คือ*ตารางเซ็กเมนต์ (segment table)* ดังรูป 7.30




รูปที่ 7.29: มุมมองโปรแกรมของนักพัฒนา




รูปที่ 7.30: การแบ่งส่วนหน่วยความจำด้วยฮาร์ดแวร์

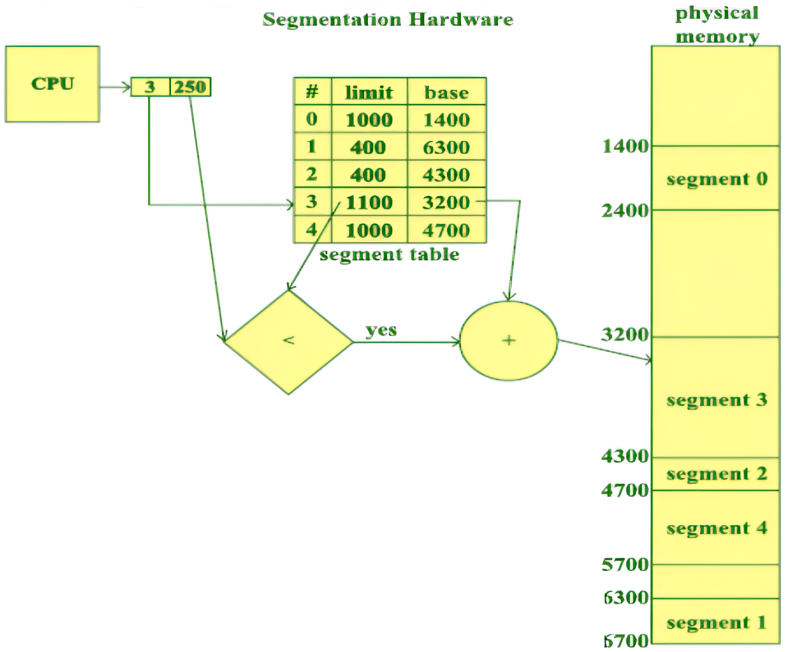
โดยเอนทรีของตารางเซ็กเมนต์ โดยทั่วไปประกอบด้วยค่าฐาน (base) และค่าขอบเขต (limit):

- *base* ชี้ไปยังตำแหน่งเริ่มต้นของส่วนในหน่วยความจำกายภาพ
- *limit* ระบุความยาวสูงสุดที่อนุญาตของส่วน (ขนาดของเซ็กเมนต์)

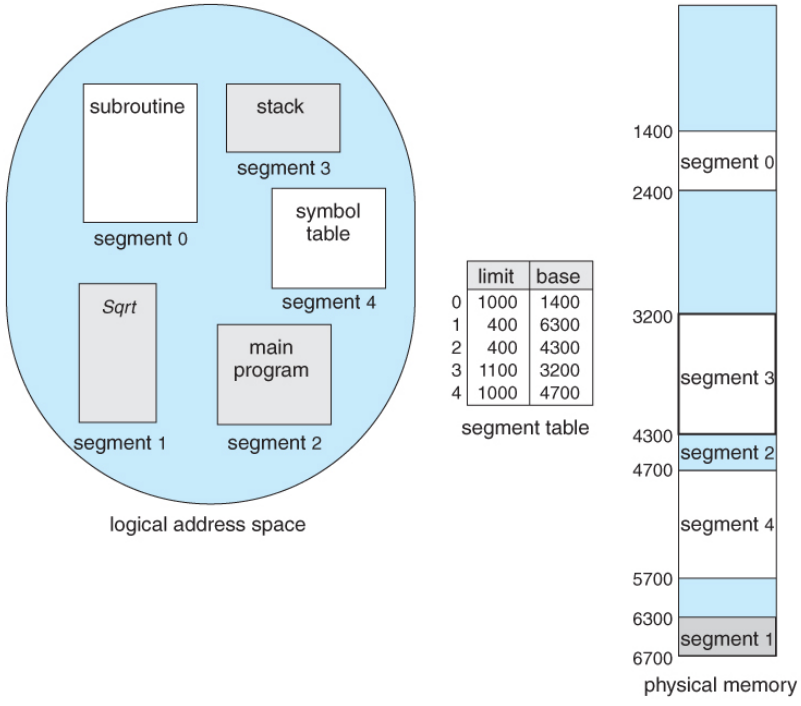
กลไกฮาร์ดแวร์แสดงในรูป 7.30: เมื่ออ้างอิง $\langle s, d \rangle$ ระบบตรวจสอบก่อนว่า $d < \text{limit}[s]$ หากไม่จริง ให้รายงานข้อผิดพลาดการอ้างอิงเกินขอบเขต มิฉะนั้นตำแหน่งจริงคือ $\text{base}[s] + d$

ตัวอย่างตามรูป 7.31: เลขส่วนที่ 3 มี offset $= 250$ ซึ่งไม่เกิน limit $= 1100$ และมี base $= 3200$ ตำแหน่งจริงจึงเป็น $3200 + 250 = 3450$

รูป 7.32 แสดงตัวอย่างมี 5 ส่วน (หมายเลข 0–4) ถูกจัดเก็บในหน่วยความจำจริง พร้อมค่าสำคัญในตารางเลขส่วน (base, limit) ของแต่ละส่วน โดยอธิบายได้ดังนี้:




รูปที่ 7.31: ตัวอย่างการทำงานการแบ่งส่วนหน่วยความจำ




รูปที่ 7.32: ตัวอย่างการแบ่งส่วน

1. **Segment 2**: ความยาว $400$ ไบต์, base $= 4300$
   อ้างอิงไปยังไบต์ที่ $53 \Rightarrow$ ตำแหน่งจริง $= 4300 + 53 = 4353$

2. **Segment 3**: ความยาว $1000$ ไบต์, base $= 3200$

อ้างอิงไปยังไบต์ที่ $852 \Rightarrow$ ตำแหน่งจริง $= 3200 + 852 = 4052$

3. **Segment 0**: ความยาว $1000$ ไบต์, base $= 1400$
อ้างอิงไปยังไบต์ที่ $1222$ (เกินขอบเขต) $\Rightarrow$ ฮาร์ดแวร์/ระบบปฏิบัติการต้องปฏิเสธการเข้าถึง (out-of-bounds) หรือจัดการตามนโยบายที่กำหนดการเข้าถึง

### 7.5.3 การสร้างตารางเซ็กเมนต์

การแบ่งส่วนทำให้แต่ละโปรแกรมประกอบด้วยหลายส่วนได้ จึงต้องมี**ตารางเซ็กเมนต์ต่อโพรเซส** ซึ่งเก็บ base และ limit ของทุกส่วน:

- หากจำนวนส่วนมีน้อย ตารางเลขส่วนอาจเก็บในรีจิสเตอร์ความเร็วสูงทั้งตาราง เพื่อให้การจับคู่รวดเร็ว
- หากส่วนมีจำนวนมาก นิยมเก็บตารางเลขส่วนในหน่วยความจำหลักและใช้รีจิสเตอร์พิเศษ *Segment Table Base Register (STBR)* เพื่อชี้ไปยังจุดเริ่มของตาราง และใช้ *Segment Table Length Register (STLR)* ระบุจำนวนเอนทรีของตารางนั้น

ในขั้นตอนการอ้างอิง $\langle s, d \rangle$ สามารถสรุปโดยย่อได้ดังนี้:

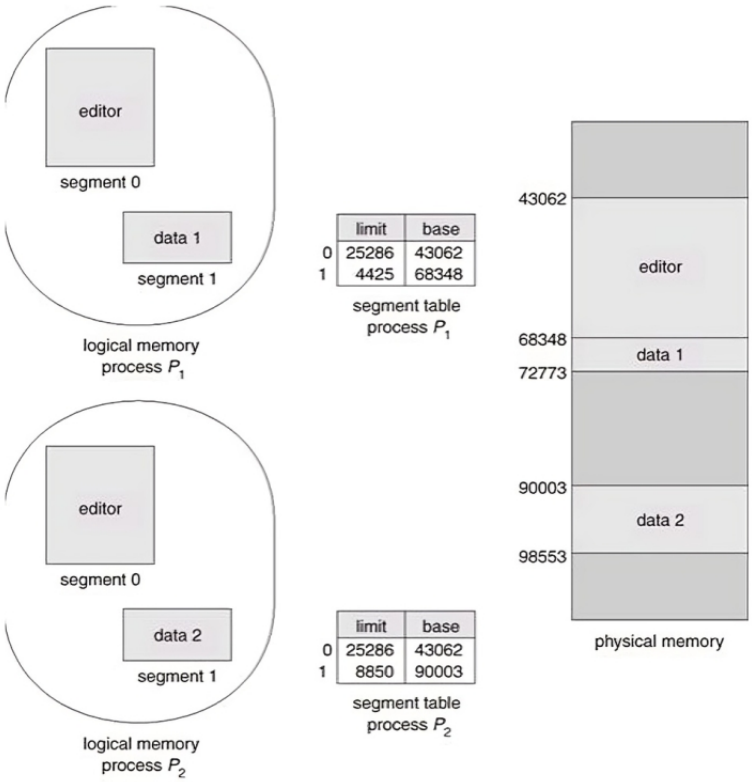
1. ตรวจว่า $s <$ STLR (หมายเลขส่วนต้องอยู่ในช่วงที่มีจริง)
2. อ่านเอนทรีตารางเลขส่วนที่ตำแหน่ง STBR $+ s$ เพื่อดึง base, limit
3. ตรวจว่า $d <$ limit แล้วคำนวณตำแหน่งจริงเป็น base $+ d$

ในวิธีการนี้ จะคล้ายกับการแบ่งเป็นหน้า การอ้างอิงหนึ่งครั้งอาจต้องอ่านหน่วยความจำ “สองครั้ง” (อ่านเอนทรีตาราง + อ่านข้อมูลจริง) ทำให้ช้าลงจึงนิยมใช้รีจิสเตอร์แคช/บัฟเฟอร์บางส่วนเก็บผลลัพธ์ที่เพิ่งใช้งาน เพื่อลดเวลาเฉลี่ยของการอ้างอิงลงให้เหลือเพิ่มขึ้นเพียงประมาณ (10%)–(15%) จากการอ้างอิงแบบตรง

### 7.5.4 การป้องกันและการใช้ส่วนร่วมกัน (Protection and Sharing)

ข้อดีของการแบ่งส่วนคือสามารถ*กำหนดสิทธิ์เฉพาะต่อส่วน* ได้ชัดเจน เช่น ส่วนคำสั่งกำหนดเป็นอ่านอย่างเดียว (read-only) หรือใช้รันได้อย่างเดียว (execute-only) เพื่อป้องกันการเขียนทับโดยไม่ตั้งใจ ฮาร์ดแวร์ยังช่วยตรวจสอบการอ้างอิงเกินขอบเขตของอาร์เรย์หรือสแต็กได้ด้วยผ่านการตรวจ $d <$ limit ในทุกครั้งที่เข้าถึง

*การใช้ร่วมกัน (sharing)* ทำได้ง่ายในระดับ “ส่วน” โดยโพรเซสหลายตัวสามารถให้เอนทรีของตารางเลขส่วนชี้ไปยังตำแหน่งฐานเดียวกันในหน่วยความจำกายภาพ (สำหรับส่วนที่แชร์) ในขณะที่ยังเก็บส่วนข้อมูลของตนเองแยกกัน ดังรูป 7.33 แนวทางนี้ลดการใช้หน่วยความจำหลักได้มาก โดยเฉพาะในระบบแบ่งเวลาที่ผู้ใช้หลายคนใช้งานโปรแกรมเดียวกันพร้อมกัน




รูปที่ 7.33: การป้องกันและใช้ส่วนร่วมกัน

### 7.5.5 การสูญเสียพื้นที่ย่อย (fragmentation)

ตัวจัดสรรหน่วยความจำในระบบแบ่งส่วนต้องรองรับ*ขนาดส่วนที่ไม่เท่ากัน* จึงคล้ายปัญหาการจัดสรรบล็อกขนาดแปรผัน และนิยมใช้นโยบาย *First-fit* หรือ *Best-fit* ผลข้างเคียงสำคัญคือเกิดการสูญเสียพื้นที่ย่อยภายนอก (external fragmentation) เพราะมีช่องว่างขนาดเล็กกระจายอยู่หลายจุดจนไม่พอสำหรับส่วนใหม่ ระบบอาจต้อง*ปิบอัดหน่วยความจำ (compaction)* เพื่อรวมช่องว่างให้ต่อเนื่องพอ

ระดับความรุนแรงของปัญหานี้ขึ้นอยู่กับ *ขนาดเฉลี่ยของส่วน*: หากมีเพียงส่วนเดียวต่อโพรเซส ระบบจะกลับไปเหมือนการจัดสรรบล็อกขนาดแปรผัน ซึ่งเกิด external fragmentation ได้มาก ตรงกันข้าม หากแยกเป็นส่วนย่อย ๆ จำนวนมากเกินไป แม้จะลด external fragmentation ได้ แต่ต้องจ่ายต้นทุนด้าน*เมตาดาทา* (ต้องมี base/limit ให้ทุกส่วน) ทำให้ใช้พื้นที่ตารางมากขึ้น หากปรับ ให้ทุกส่วนมีขนาดเท่า ๆ กัน ระบบก็จะเข้าใกล้แนวคิดของการแบ่งเป็นหน้านั่นเอง

โดยทั่วไป ถ้าส่วนมีขนาดเฉลี่ยเล็กลง ช่องว่างภายนอกโดยรวมก็มักเล็กลงด้วย เพราะมีความยืดหยุ่นสูงขึ้นในการบรรจุส่วนลงพื้นที่ว่างหลายรูปแบบ อย่างไรก็ดี ต้องสมดุลกับค่าใช้จ่ายของการเก็บและจัดการตารางเลขส่วน เพื่อไม่ให้ต้นทุนด้านเมตาดาทาและเวลาอ้างอิงสูงเกินไป

## คำถามท้ายบท

1. อธิบายความแตกต่างระหว่าง internal fragmentation และ external fragmentation
2. บอกข้อดี–ข้อเสียของการใช้ segmentation เทียบกับ paging
3. อธิบายบทบาทของ TLB (Translation Lookaside Buffer) ในระบบที่ใช้ paging
4. ถ้าระบบปฏิบัติการเปลี่ยนจาก segmentation เพียงอย่างเดียว → ไปใช้ segmentation with paging จะเกิดประโยชน์และปัญหาใหม่ ๆ อย่างไรบ้าง?
5. อธิบายวิธีการที่ segmentation สามารถช่วยสนับสนุน การป้องกันหน่วยความจำ (memory protection) และการแชร์หน่วยความจำ (memory sharing) ระหว่างโปรเซส
6. ทำไมสถาปัตยกรรมคอมพิวเตอร์สมัยใหม่ส่วนใหญ่เลือกใช้ paging เป็นหลักมากกว่า segmentation